Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models

Qizheng Zhang

PhD candidate, Stanford University

ICLR 2026; Joint work with Stanford, SambaNova Systems, UC Berkeley

Findings with Tool-Using Coding Agents

- **Case study:** the AppWorld benchmark
- Coding agent with an interactive environment
- Multi-turn, long-context tasks: Many task queries > 10 turns and > 32K tokens
- Featuring customized tool and API definitions for problem solving

Instruction: Give me a comma-separated list of top 4 most played r&b song titles from across my Spotify song, album and playlist libraries.

Supervisor:
→ **Name:** Glenn Burton
→ **Email:** glenn.burton@gmail.com
→ **Phone:** 8638518861

Datetime: 2023-05-18T12:00:00

 User: Find me some songs!

Findings with Tool-Using Coding Agents

- **Case study:** the AppWorld benchmark
- Coding agent with an interactive environment
- Multi-turn, long-context tasks: Many task queries > 10 turns and > 32K tokens
- Featuring customized tool and API definitions for problem solving

```
[
  { "name": "show_account",
    "description": "Show your account information. Unlike show_profile, this includes private information." },
  { "name": "signup",
    "description": "Sign up to create account." },
  …
]
```



Agent: Cool, I can use these

Findings with Tool-Using Coding Agents

- **Case study:** the AppWorld benchmark
- The agent often forgets to retriever a user's access token before further operations.

```
access_token = login_result['access_token']
page_index = 0 songs = []
while True:
    song_page = apis.spotify.show_song_library(access_token=access_token, page_index=page_index)
    if song_page:
        songs.extend(song_page)
        page_index += 1
    else:
        break
print(songs)
```

 Agent: Let me attempt by writing code

Findings with Tool-Using Coding Agents

- **Case study:** the AppWorld benchmark
- The agent often forgets to retriever a user's access token before further operations.

```
access_token = login_result['access_token']
page_index = 0 songs = []
while True:
    song_page = apis.spotify.show_song_library(access_token=access_token, page_index=page_index)
    if song_page:
        songs.extend(song_page)
        page_index += 1
    else:
        break
print(songs)
```

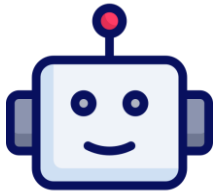
❌ Error! Key "access_token" does not exist!

Findings with Tool-Using Coding Agents

- **Case study:** the AppWorld benchmark
- What we did: enforce access token retrieval as a first step in the agent context.



How about enforcing this rule in context?
Let's manually write something like "principle 1: access token …"



Principle 1: Always log in to retrieve access token from users as a first step.

System Prompt: You are an LLM agent that …
Tools to be used: …

Findings with Tool-Using Coding Agents

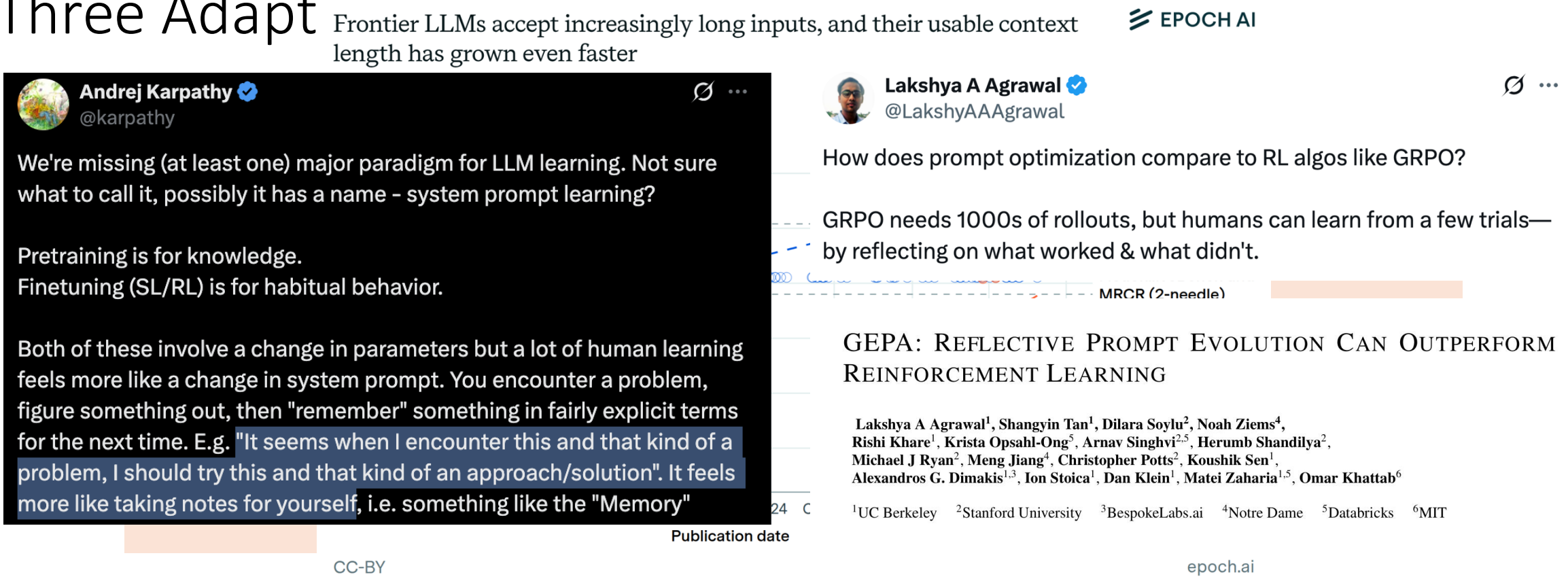
- **Case study:** the AppWorld benchmark
- The agent often forgets to retriever a user's access token before further operations.

```
login_result = apis.spotify.login(username='glenn.burton@gmail.com', password=spotify_password)
print(login_result)

{
   "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzcG90aWZ5K2dsZW5uLmJ1cnRvbkBnbWFpbC5jb20iLCJleHAi
OjE2ODQ0MTI1NzN9.2mg3kxnPgfHlBzAJMmFCN0qFJImttY4NSSnQb3w5o78",
   "token_type": "Bearer"
}
```

Agents often miss *contextual/procedural knowledge*, instead of facts.

Three Adapt



Context evolution (this talk)

Pros: fast test-time learning, interpretable, model-agnostic

Cons: needs stability + cost control at long horizons (next few slides)

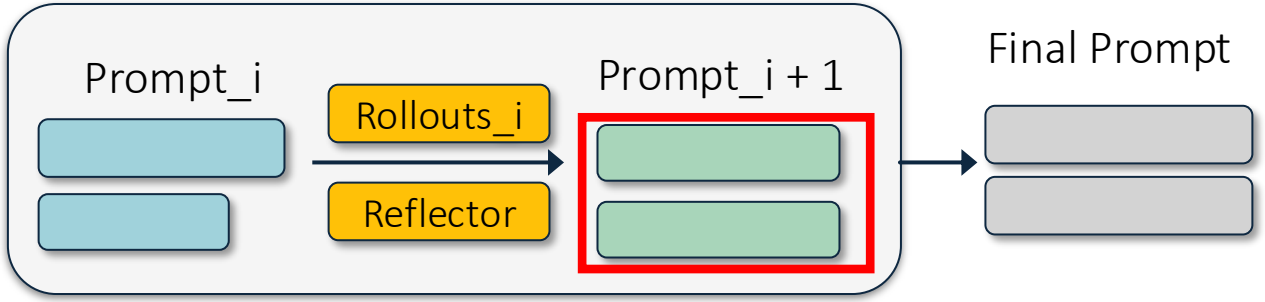
Limitations of Existing Context Adaptation Methods

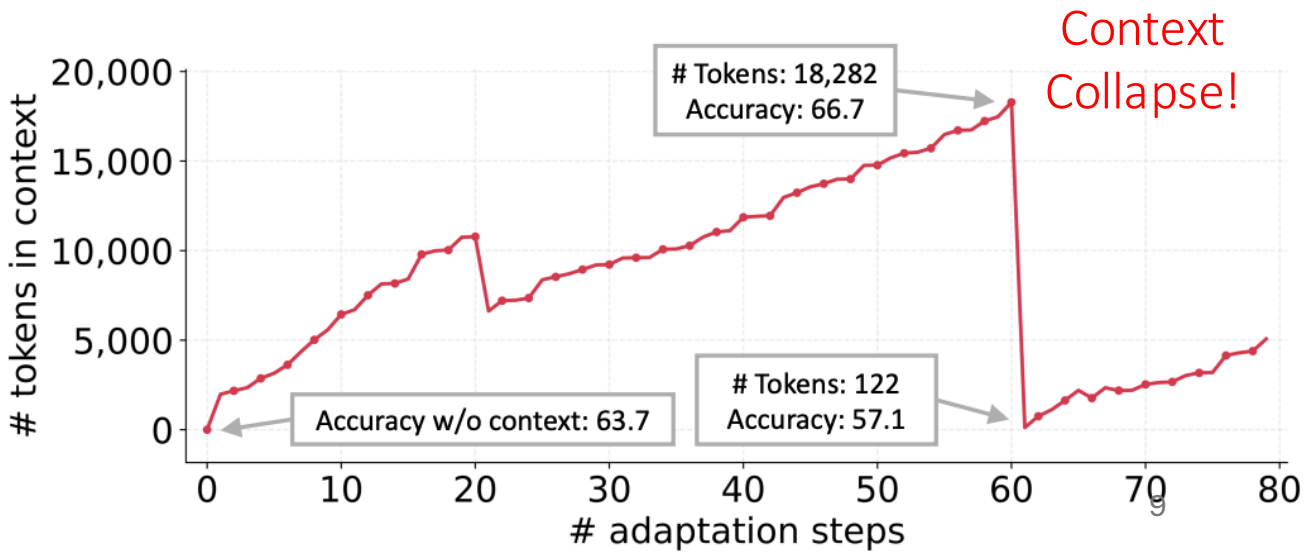
Two limitations

- Brevity bias
  - A preference for short, concise, and generic instructions
- Context collapse
  - Rewriting collapse contexts into shorter, less informative summaries

Existing Methods

GEPA (prompt optimizers), Dynamic Cheatsheet (agent memory), etc.





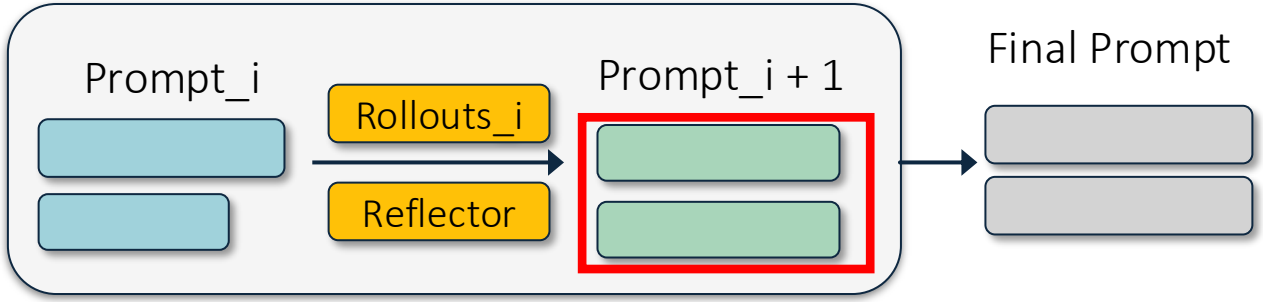
Limitations of Existing Context Adaptation Methods

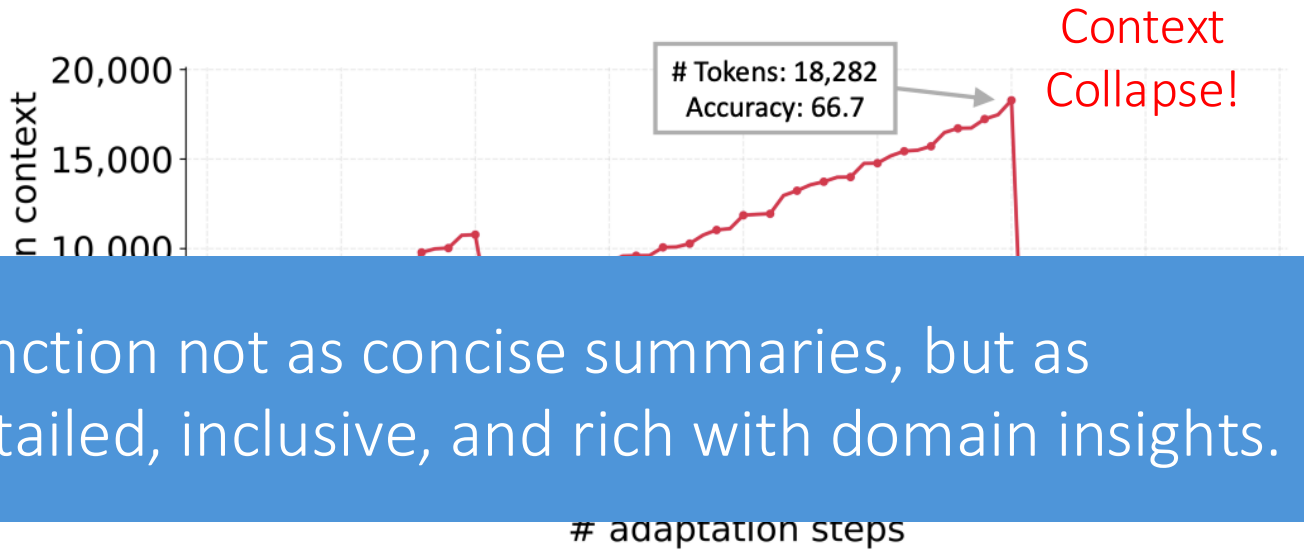
Two limitations

- Brevity bias
  - A preference for short, concise, and generic instructions
- Context collapse
  - Rewriting collapse contexts into shorter, less informative summaries

Existing Methods

GEPA, Dynamic Cheatsheet, etc.

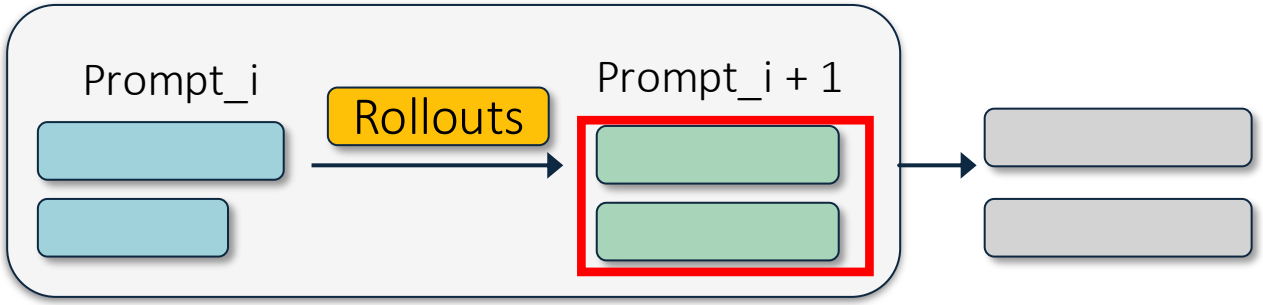




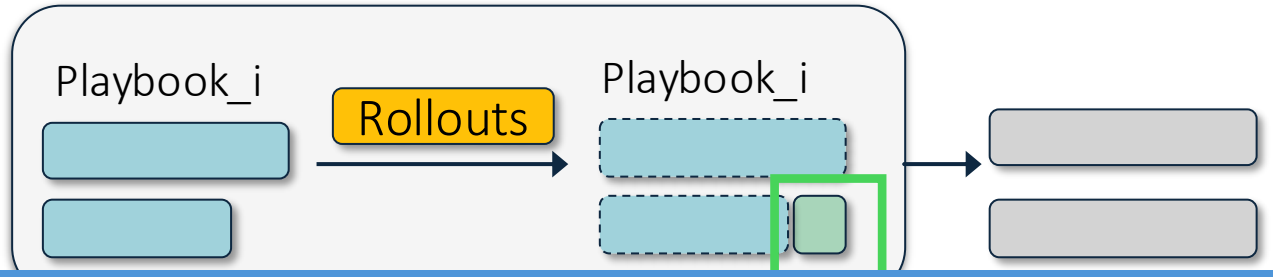
Our proposal: Contexts should function not as concise summaries, but as comprehensive, evolving playbooks—detailed, inclusive, and rich with domain insights.

Incremental Context Update

Prior: monolithic rewrite → collapse

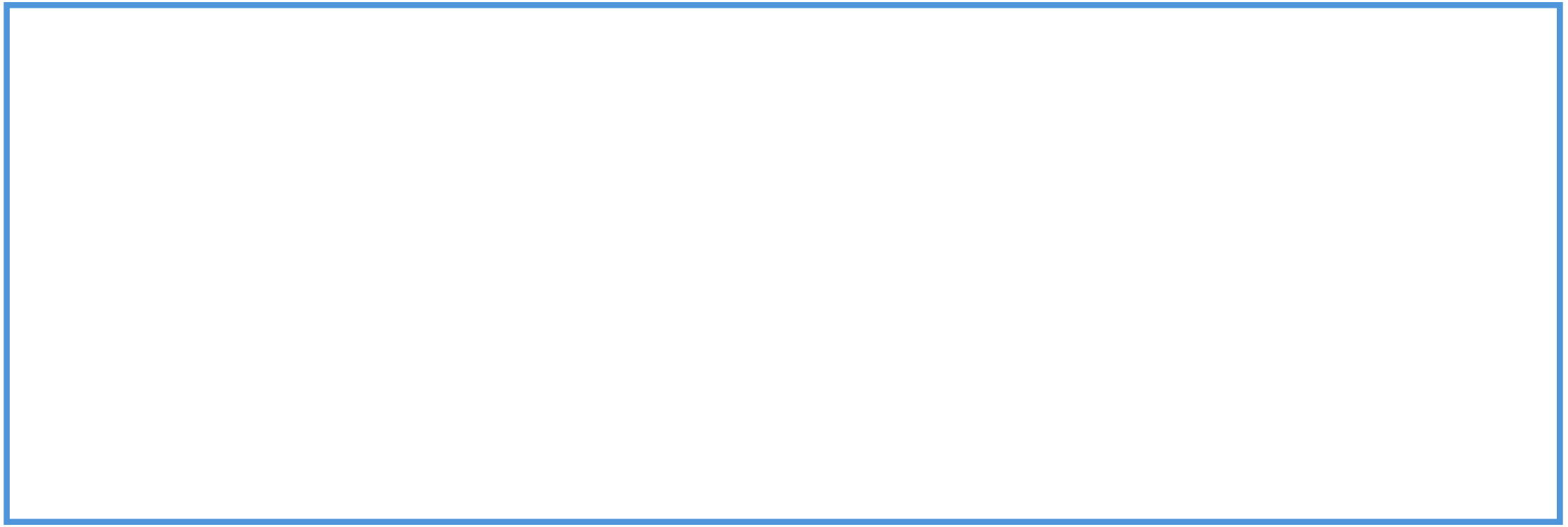


Ours: delta updates → stable growth

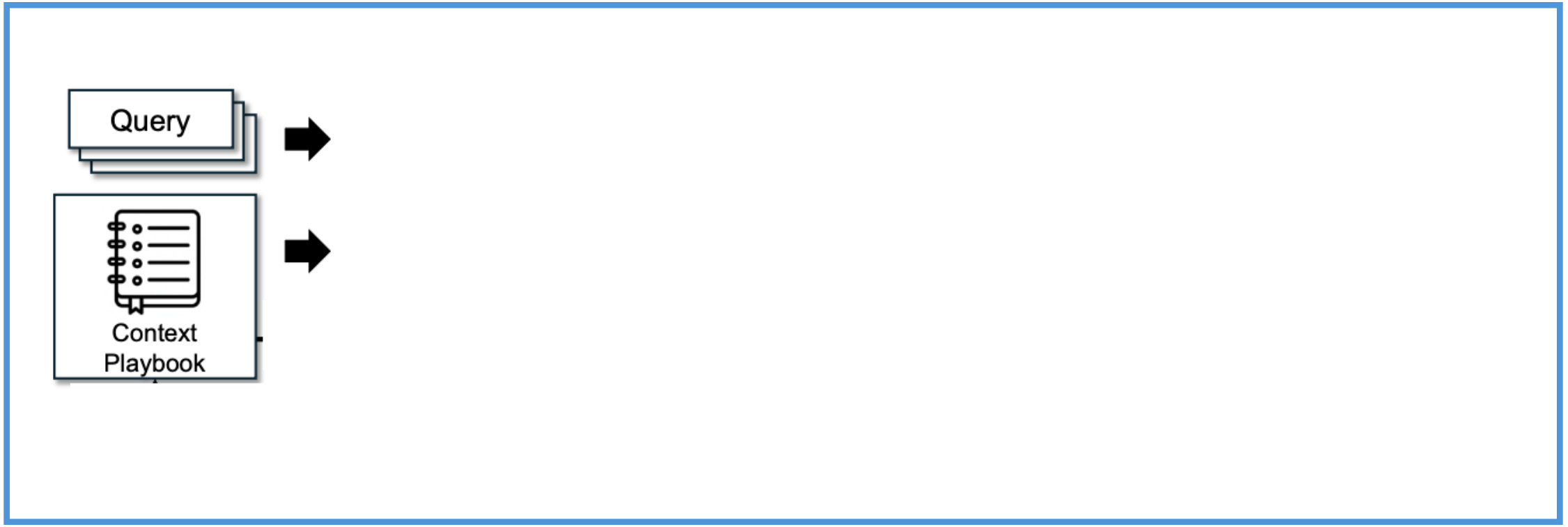


Recipe #1: Incremental update for graceful scaling of model context

ACE (Agentic Context Engineering)



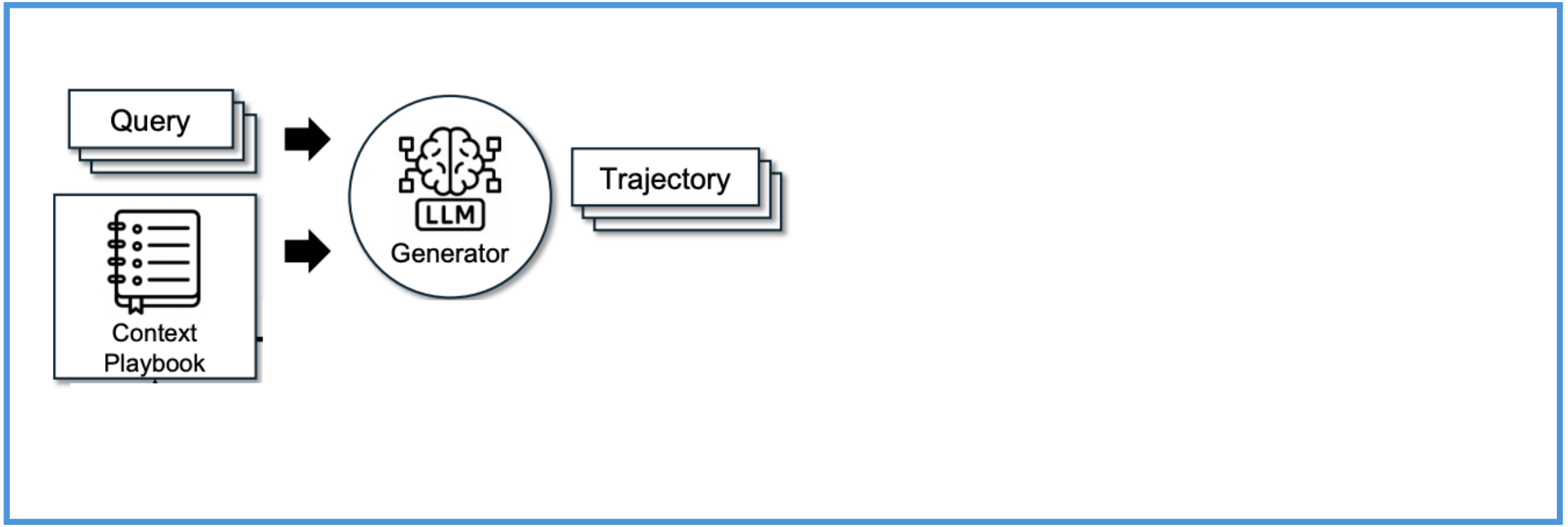
ACE (Agentic Context Engineering)



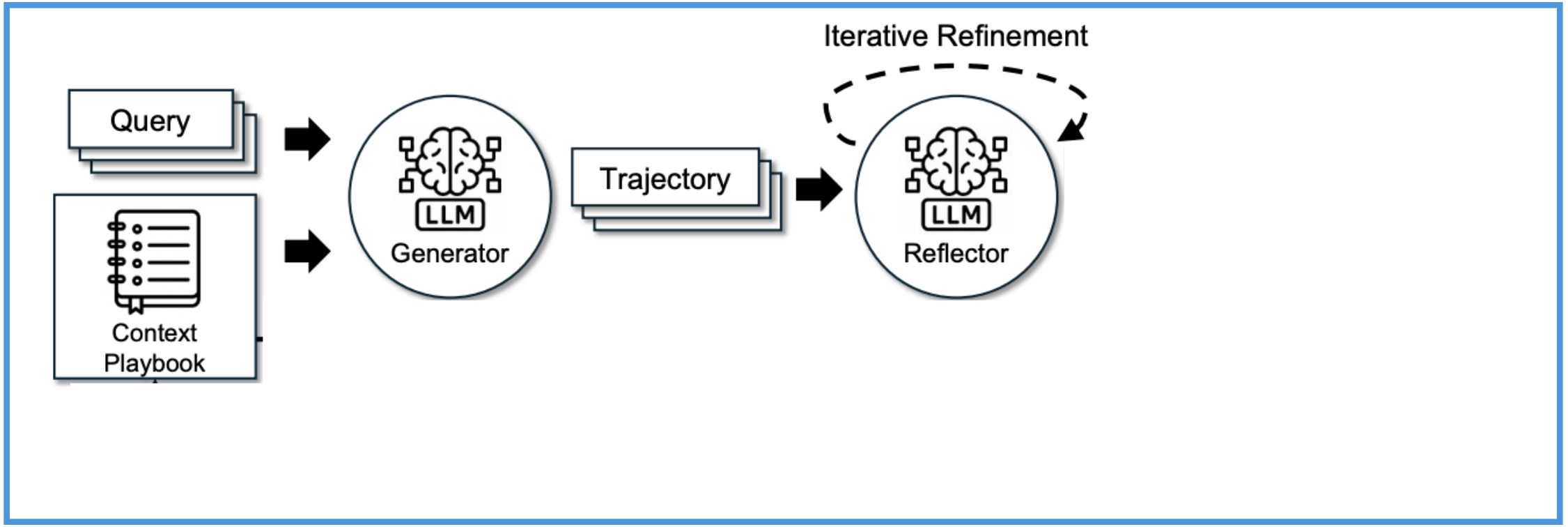
ACE (Agentic Context Engineering)



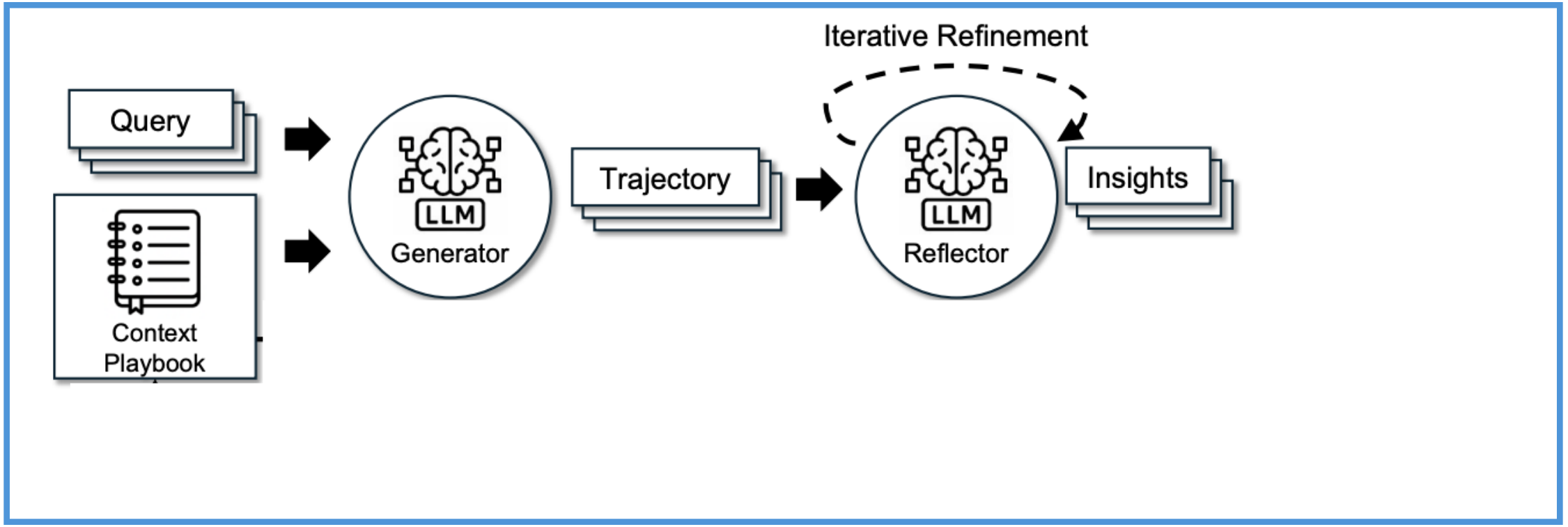
ACE (Agentic Context Engineering)



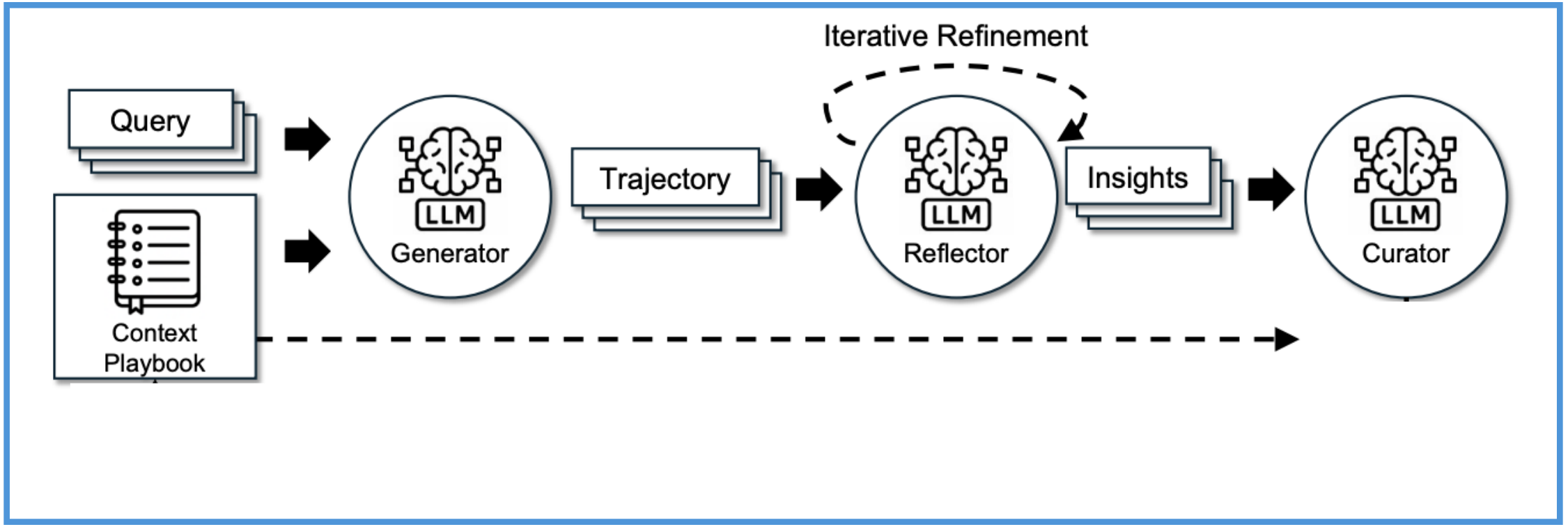
ACE (Agentic Context Engineering)



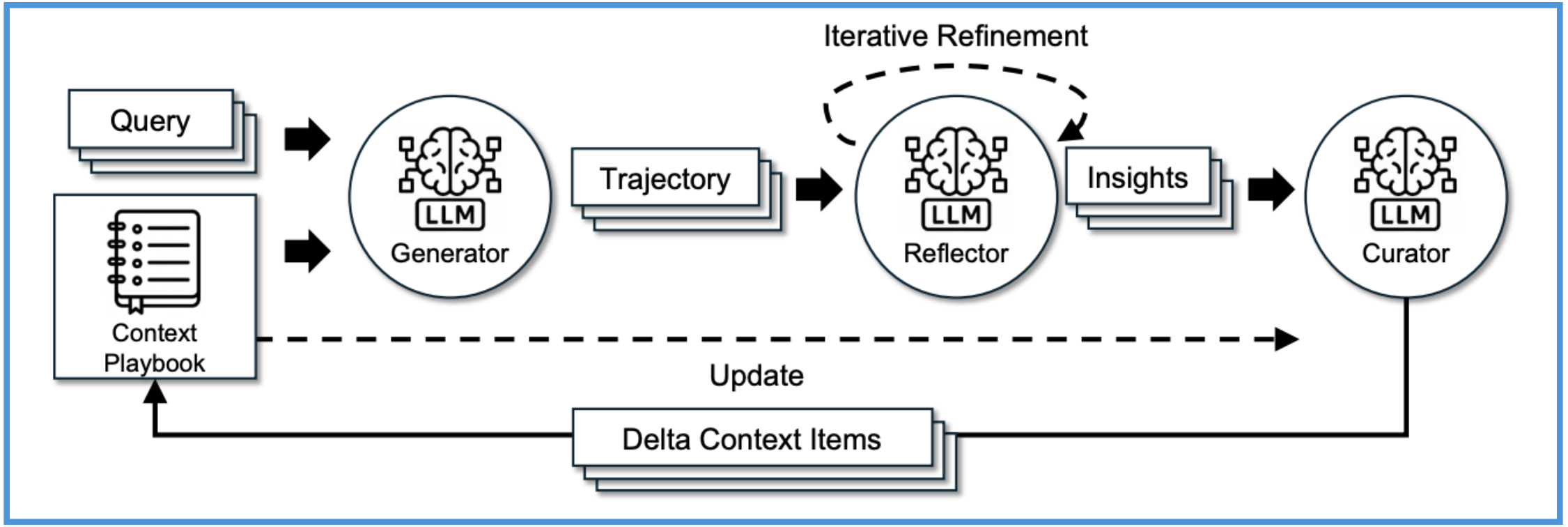
ACE (Agentic Context Engineering)



ACE (Agentic Context Engineering)



ACE (Agentic Context Engineering)



Recipe #2: Structured division of labor (Generator, Reflector, Curator)

Isn’t this just prompt engineering?

Formalizing Context Evolution as a System Problem

- With ACE, we have a ***state*** and an ***update operator*** over that state, under an autonomous agent loop ***architecture***

- State: a better ***representation*** of evolving context as first-class citizen
  - Sections
  - Itemized bullets + metadata → localization, fine-grained retrieval, incremental adaptation, periodic de-duplication and clean-up
- Update operator: delta updates + deterministic merge
  - Compact delta contexts instead of full rewrites → preserves old knowledge, avoids collapse, enables batching/parallel learning
- Architecture: autonomous loop + structured division of labor
  - Separates execution, insight extraction, and integration (and allows iterative refinement)

Case Study: AppWorld (Agent)

Pre-defined general section names

Generated bullet-points of insights

ACE-generated contexts contain detailed, domain-specific insights along with tools and code that are readily usable, serving as a comprehensive playbook for LLMs.

The Generated ACE Playbook on AppWorld

STRATEGIES AND HARD RULES

[shr-00009]

When processing time-sensitive transactions involving specific relationships: always resolve identities from the correct source app (phone contacts), use proper datetime range comparisons instead of string matching, and verify all filtering criteria (relationship + time) are met before processing items. This ensures accurate identification and processing of the right transactions.

USEFUL CODE SNIPPETS AND TEMPLATES

[code-00013]

For efficient artist aggregation when processing songs, use defaultdict(list) to map song titles to artist names:

```
from collections import defaultdict artist_map = defaultdict(list) for song in songs:
artist_map[song['title']].extend([artist['name'] for artist in song['artists']])
```

TROUBLESHOOTING AND PITFALLS

[ts-00003]

If authentication fails, troubleshoot systematically: try phone number instead of email as username, clean credentials from supervisor, check API documentation for correct parameters etc. Do not proceed with workarounds.

ACE-generated playbook on AppWorld (partially shown)

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	$64.3_{+0.6}$	$46.4_{+3.5}$	$46.0_{+4.5}$	$27.3_{+5.7}$	$46.0_{+3.6}$
ReAct + GEPA	✓	$64.9_{+1.2}$	$44.6_{+1.7}$	$46.0_{+4.5}$	$30.2_{+8.6}$	$46.4_{+4.0}$

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	64.3$_{+0.6}$	46.4$_{+3.5}$	46.0$_{+4.5}$	27.3$_{+5.7}$	46.0$_{+3.6}$
ReAct + GEPA	✓	64.9$_{+1.2}$	44.6$_{+1.7}$	46.0$_{+4.5}$	30.2$_{+8.6}$	46.4$_{+4.0}$
ReAct + ACE	✓	**76.2**$_{+12.5}$	**64.3**$_{+21.4}$	**57.3**$_{+15.8}$	**39.6**$_{+18.0}$	**59.4**$_{+17.0}$

ACE achieves better performance than RAG-based ICL and prompt optimizers.

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	64.3$_{+0.6}$	46.4$_{+3.5}$	46.0$_{+4.5}$	27.3$_{+5.7}$	46.0$_{+3.6}$
ReAct + GEPA	✓	64.9$_{+1.2}$	44.6$_{+1.7}$	46.0$_{+4.5}$	30.2$_{+8.6}$	46.4$_{+4.0}$
ReAct + ACE	✓	**76.2**$_{+12.5}$	**64.3**$_{+21.4}$	**57.3**$_{+15.8}$	**39.6**$_{+18.0}$	**59.4**$_{+17.0}$
ReAct + ACE	✗	75.0$_{+11.3}$	**64.3**$_{+21.4}$	54.4$_{+12.9}$	35.2$_{+13.6}$	57.2$_{+14.8}$

ACE can evolve context even without supervised labels, using environment feedback.

Case Study: AppWorld (Agent)

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	64.3$_{+0.6}$	46.4$_{+3.5}$	46.0$_{+4.5}$	27.3$_{+5.7}$	46.0$_{+3.6}$
ReAct + GEPA	✓	64.9$_{+1.2}$	44.6$_{+1.7}$	46.0$_{+4.5}$	30.2$_{+8.6}$	46.4$_{+4.0}$
ReAct + ACE	✓	**76.2**$_{+12.5}$	**64.3**$_{+21.4}$	**57.3**$_{+15.8}$	**39.6**$_{+18.0}$	**59.4**$_{+17.0}$
ReAct + ACE	✗	75.0$_{+11.3}$	**64.3**$_{+21.4}$	54.4$_{+12.9}$	35.2$_{+13.6}$	57.2$_{+14.8}$
Online Adaptation						
ReAct + DC (CU)	✗	65.5$_{+1.8}$	**58.9**$_{+16.0}$	52.3$_{+10.8}$	30.8$_{+9.2}$	51.9$_{+9.5}$
ReAct + ACE	✗	**69.6**$_{+5.9}$	53.6$_{+10.7}$	**66.0**$_{+24.5}$	**48.9**$_{+27.3}$	**59.5**$_{+17.1}$

ACE could evolve context both offline (system prompt) and online (agent memory).

Lower Cost, Better Efficiency

- We did a component-wise cost analysis for ACE and GEPA (prompt optimizer).

Method	Total # input tokens	Total # output tokens	Total # rollouts	Total # queries
GEPA (prompt generation)	65,005,192	1,266,090	429	90
GEPA (prompt validation)	139,070,904	604,098	1,026	57
GEPA (total)	204,076,096	1,870,188	1,455	147
ACE (Generator)	31,012,122	198,847	1,790	90
ACE (Reflector)	4,685,840	70,487	161	90
ACE (Curator)	3,552,963	37,794	124	90
ACE (total)	39,250,925 (-80.8%)	307,128 (-83.6%)	2,075 (+42.6%)	90 (-38.8%)

Method	Latency (s)↓	# Rollouts↓
ReAct + GEPA	53898	1434
ReAct + ACE	9517 (-82.3%)	357 (-75.1%)

(a) **Offline** (AppWorld).

ACE significantly reduces tokens usage and adaptation latency as compared to GEPA.

KV Cache Reuse: Longer Context ≠ Higher Serving Cost

- We did a component-wise cost analysis for ACE and GEPA (prompt optimizer).
- ACE uses more raw input tokens, but 91.8% of them can be served from KV-cached prompts, reducing 82.6% billed input token cost.

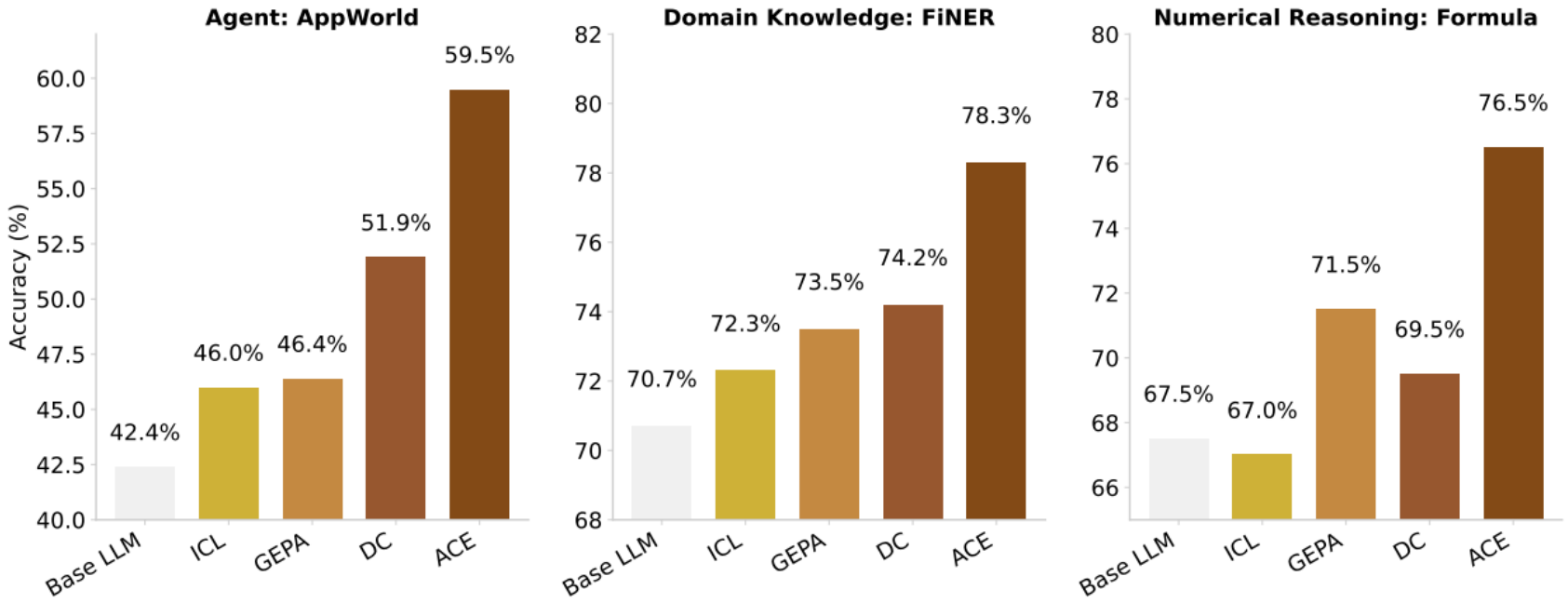
Method	Total # input tokens	Total # output tokens	Total # rollouts	Total # eval queries
Base ReAct	27,460,411	289,802	2,430	160
GEPA	26,960,675	251,442	2,470	160
ACE	58,623,267 (+117.4%)	270,652 (+7.6%)	2,354 (-4.7%)	160 (+0.0%)

ACE uses more tokens during inference, but KV cache reuse helps to reduce cost.

Summary of Results

Accuracy: +10.6% on agents and +8.6% on finance;
Efficiency: significantly reducing adaptation latency and rollout cost.

Overall Application Performance



Efficiency

Method	Latency (s)↓	# Rollouts↓
ReAct + GEPA	53898	1434
ReAct + ACE	9517 (-82.3%)	357 (-75.1%)

(a) **Offline** (AppWorld).

Method	Latency (s)↓	Token Cost ($)↓
DC (CU)	65104	17.7
ACE	5503 (-91.5%)	2.9 (-83.6%)

(b) **Online** (FiNER).

ACE Used in Production at SCX.ai

SCX.ai is an AI product and research company in Australia; https://labs.scx.ai/ace

- Customer service workflow (chatbot)

```
Customer: "Hi, I'm contacting you because package damaged on delivery."
```

- **Without ACE:** the chatbot always issues a refund (which is funny yet dangerous)

No playbook

```
Sorry to hear that your package was damaged on delivery. I've
escalated this issue to our priority team to ensure it's resolved
within the next 24 hours. You will receive a replacement or a full
refund…
```

ACE Used in Production at SCX.ai

SCX.ai is an AI product and research company in Australia; https://labs.scx.ai/ace

- Customer service workflow (chatbot)

```
Customer: "Hi, I'm contacting you because package damaged on delivery."
```

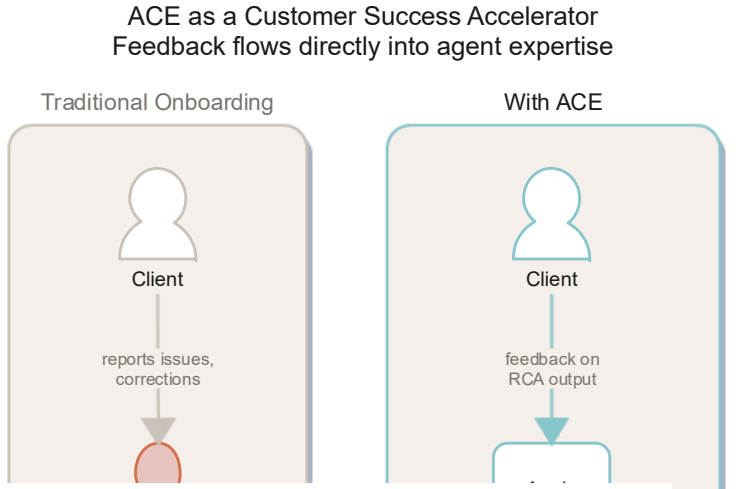
- **With ACE:** the chatbot always asks for and verifies order ID + order details
  - With ACE playbook

```
Sorry to hear that your package was damaged on delivery. I've
located your account and I'm here to help. Could you please
provide your order ID so I can verify the details?
```

ACE Used in Production at AnyShift

Anyshift is a platform engineering startup company in France;

https://www.anyshift.io/blog/ace-retrieval-layer



Production Impact

30% faster

Root cause analysis time reduction in production

Statistically significant improvements

Internal evaluations measured gains across three dimensions:

Tool use proficiency: agents pick the right tools faster

Correctness: fewer false leads and misidentifications

Impact analysis: more complete blast radius assessment

Where We See This Going

- We can treat context as a fast, test-time ***medium*** for continual learning, and then selectively distill it into weights when it stabilizes.
- Parallel context learning at scale: Combee, COLM 2026
- Meta-evolution for memory systems and harnesses: Meta-Harness, COLM 2026
- Safety + correctness of updates (ML + systems)
- Consolidation (learning): In-context or weight-update self-distillation

Conclusion

- Context evolution as a new paradigm for AI adaptation
- The brevity bias and context collapse in existing methods
- Introducing ACE (Agentic Context Engineering)
- Enabling self-improving agents and domain AI with ACE

- Join the ACE community slack!
- Paper: https://arxiv.org/pdf/2510.04618
- Code: https://github.com/ace-agent/ace
- Blogs: https://ace-agent.github.io/blogs/
- Contact: qizhengz@stanford.edu